

ASTE404 Mini Project – Electric Propulsion Optimization Tool

By: Faith Colon

Outline:

- I. Project Overview*
- II. Numerical Methods*
- III. Implementation*
- IV. Verification*
- V. Results Example + Reflection*
- VI. Progress Log*
- VII. Co-Pilot LLM Transcript*

I. -----PROJECT OVERVIEW -----

PROJECT GOALS:

Primary goal is to find the optimal design for an EP thruster under fixed mission requirements by varying parameters.

1. Mission Requirements:

- Reasonable $\Delta V = 9500 \frac{m}{s}$
- Burn time $T_b = 200$ days

2. Maximize the payload ratio $\frac{M_L}{M_0}$ using following equation

$$\frac{M_L}{M_0} = e^{-\frac{\Delta V}{v_e}} \left(1 + \frac{\alpha v_e^2}{2\eta t_b} \right) - \frac{\alpha}{\eta} \frac{v_e^2}{2t_b} \quad \text{Equation 1}$$

- Use Optimization tool: find maximum value of M_L/M_0 & report ISP

3. Consider multiple configurations and test their stability:

- Thruster configurations varying specific mass α and efficiency η

Essentially: My initial goal was to replicate the following figure, and find optimal isp point (highest payload ratio) for specific thruster:

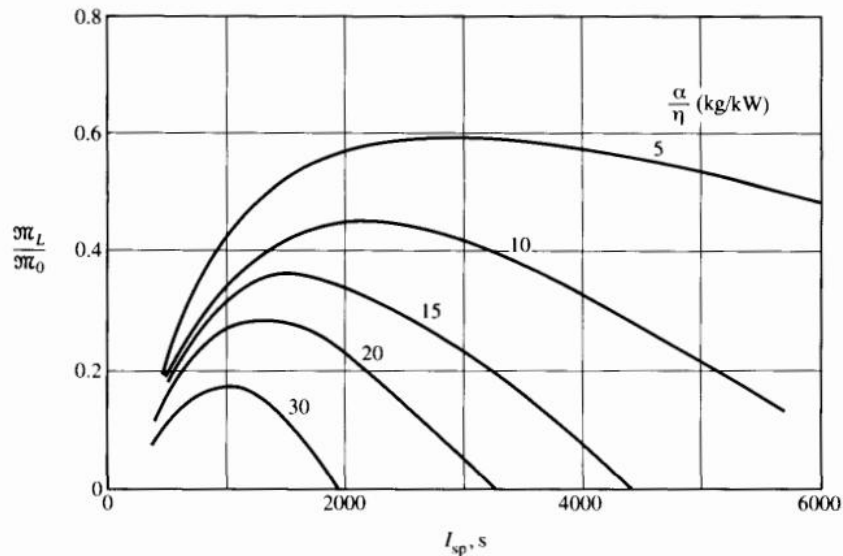


FIGURE 10.11 Variation of payload ratio with specific impulse according to Eq. (10.43) for a one-month earth-moon transfer.

Fig. 1 – Figure from Hill & Peterson’s “*Mechanics and Thermodynamics of Propulsion*”
Textbook pg. 493

Additional notes:

- Main Results:
 1. **Values of Maximum payload ratio achievable, and what ISP is required for it, as well as optimal α and η values.**
 2. **Plot of $\frac{M_L}{M_0}$ vs. ISP**
- “Optimal configuration” in this project primarily means **maximum payload ratio**
- Plots enable comparison of multiple thruster configurations and identify globally optimal parameters

II. ----- NUMERICAL METHODS -----

1. Optimization algorithm: L-BFGS-B (Limited-memory Broyden-Fletcher-Goldfarb-Shanno with Bounds) – finds minimum value

→ used this find maximum payload ratio $\frac{M_L}{M_0}$ over a range of ISP values

- **Notes on: L-BFGS-B algorithm:**
 - Limited-memory Broyden–Fletcher–Goldfarb–Shanno with Box constraints.
 - A gradient-based quasi-newton method (meaning instead of calculating full 2nd derivative matrix, it uses gradients from previous steps)
 - B- using simple “boxed” constraints
 - Limited memory – not very memory intensive, runs faster/efficient
- *How implemented in code (file: **Electric_Propulsion.py**): See below*

- *Parameter Bounds: Used based on Hill & Peterson specific mass values:*

```
# Bounds for each parameter:
# Alpha (kg/W), Eta (Efficiency), Isp (s)
bounds = [
    (0.0014, 0.2), # Alpha: specific mass (1-15 g/W)
    (0.3, 0.9),    # Eta: thruster efficiency (40% to 90%)
    (100, 9000)    # Isp: Specific impulse range
]

# Initial guess for the parameters, using midpoint of bounds
initial_guess = [
    (bounds[0][0] + bounds[0][1]) / 2, # Alpha midpoint
    (bounds[1][0] + bounds[1][1]) / 2, # Eta midpoint
    (bounds[2][0] + bounds[2][1]) / 2  # Isp midpoint
]

# Use L-BFGS-B (gradient-based method with bounds) for optimization

opt_result = minimize(neg_payload, initial_guess, bounds=bounds, method='L-BFGS-B')
return opt_result
```

2. Main Equation Used

$$\frac{M_L}{M_0} = e^{-\frac{\Delta V}{v_e}} \left(1 + \frac{\alpha v_e^2}{2\eta t_b} \right) - \frac{\alpha v_e^2}{\eta 2t_b} \quad \text{Equation 1}$$

Hill & Peterson Equation 10.43 on Pg.492

- Equation Findings
 - Plots weren't behaving right (giving negative values of M_L/M_0) → (went through each variable in equation 1, found that α needs to be smaller than ~0.277
 - TLDR: Hill & Peterson has a table of α values in Kg/KW, (Table 10.5) which produced errors in graph (negative M_L/M_0 values due to too high correction factor) but switching this to units of Kg/W fixed it.
- Hill & Peterson References:

TABLE 10.5 Specific mass of nuclear power plants for space vehicles [2]

Electrical power level (kW)	Specific mass (kg/kW)
100	30.0
500	7.0
1,000	3.9
5,000	1.9
10,000	1.6

-

$$\alpha = \frac{M_p + M_s}{P},$$

-

Specific Mass Equation 10.39

- **Limitations:**

- Can find local optima (not guaranteed global)

III. ----- IMPLEMENTATION -----

Implementation notes: Key design choices, file/directory layout, how to run (similar to README)

(See README file on Github)

- Used Python (simple, easy to make plots)
- Navigate to file:
 - **cd rocket-relations\src\rocket_relations**
- The main files are located in **src/rocket_relations/**
 - electric_propulsion.py
 - interactive_tool.py
- **electric_propulsion.py** - has core functions
- **interactive_tool.py** - main interactive file – RUN this one

IV. ----- Verification/Validation -----

- Tested bounds of alpha, eta values to break code (i.e return negative Payload Mass requirements)
 - Limit → very low alpha ($.0014 (m_{PP} + m_s/P)$) requires super high isp, means PP and structure mass are too low
- Mostly time was spent running code, giving it values at the boundaries, essentially trying to “break” the code and then fix the errors. (Est Time: 4-5 hours for 2-3 days at a time)
- Ensured that when something returns “negative”, it gives a warning and suggests alternatives
- Also, note that sometimes “Optimal Isp is 9000s”, which is clearly the upper bound that I set. This just means that the mission requires extremely high performance, OR the structural mass (alpha) is too high.

V. ----- RESULTS -----

Method A: Single inputs for specific mass and efficiency. Code outputs optimal isp and payload ratio achieved. Plots results.

```
(base) C:\Users\faith\ASTE404\Homework5\rocket-relations\src\rocket_relations>python interactive_tool.py
Welcome to the Electric Propulsion Interactive Tool!
This tool will guide you through analyzing thruster configurations.

Please adhere to the suggested input bounds.

Enter the required Delta-V (m/s) [range: 5000-15000]: 7000
Enter the mission duration (burn time, in days) [range:20-200]: 150

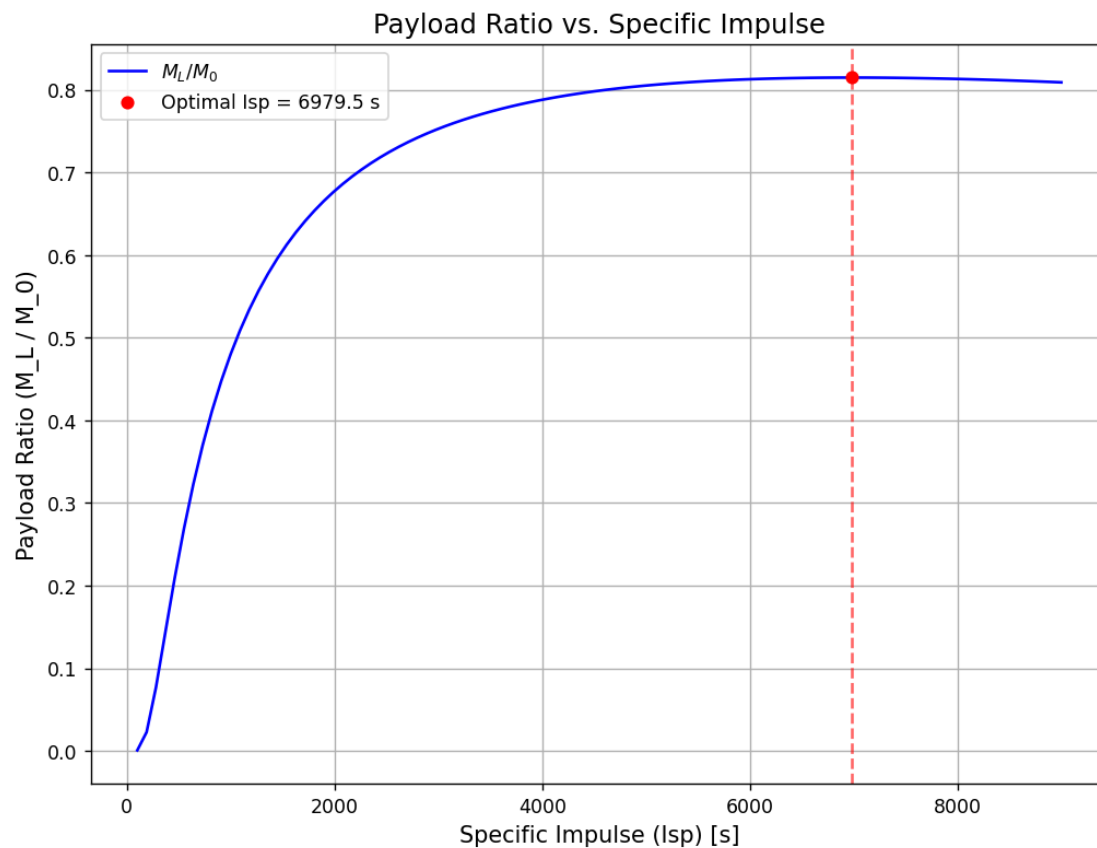
Choose a method for thruster analysis:
A: Optimize for a single configuration (user chooses 1 specific mass and efficiency value)
B: Optimize for multiple configurations (user chooses multiple specific mass and efficiency values)
C: Global optimization for mission requirements

Enter 'A', 'B', or 'C': A

--- Single Configuration ---
Enter structural mass ratio  $\alpha$  (Kg/W, range: 0.0015-0.266): 0.003
Enter thruster efficiency  $\eta$  (value between 0 and 1): 0.6

Results for Single Thruster Configuration:
  Optimal Specific Impulse (Isp): 6979.48 seconds
  Maximum Payload Ratio: 0.81493
```

```
Results for Single Thruster Configuration:
  Optimal Specific Impulse (Isp): 6979.48 seconds
  Maximum Payload Ratio: 0.81493
```



Method B: User can input multiple alpha and eta values. Code computes optimal configuration of given values & Plots them.

```
Welcome to the Electric Propulsion Interactive Tool!
This tool will guide you through analyzing thruster configurations.

Please adhere to the suggested input bounds.

Enter the required Delta-V (m/s) [range: 5000-15000]: 15000
Enter the mission duration (burn time, in days) [range:20-200]: 150

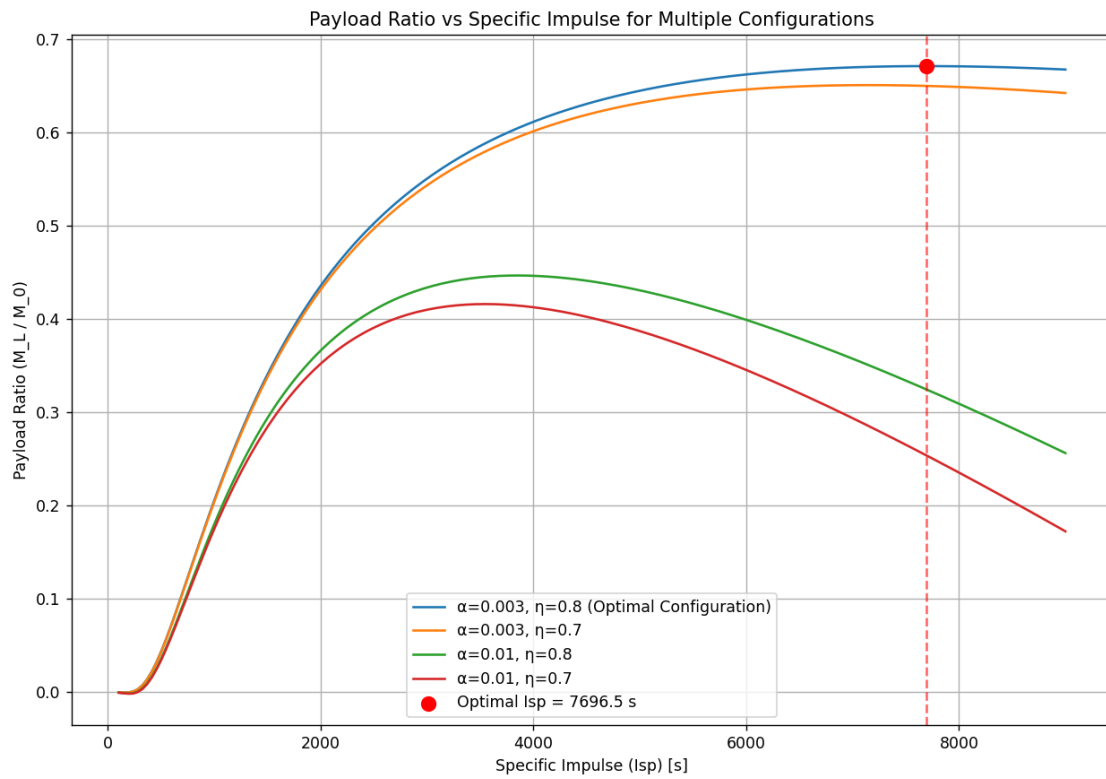
Choose a method for thruster analysis:
A: Optimize for a single configuration (user chooses 1 specific mass and efficiency value)
B: Optimize for multiple configurations (user chooses multiple specific mass and efficiency values)
C: Global optimization for mission requirements

Enter 'A', 'B', or 'C': B
Enter alpha values (comma-separated, within range of 0.0015 to 0.266 g/W): 0.003,0.01
Enter eta values (comma-separated, e.g., 0.5,0.8): 0.8,0.7

--- Best Configuration from user inputs ---
Best Alpha ( $\alpha$ ): 0.00300 kg/W
Best Eta ( $\eta$ ): 0.8000
Best Isp: 7696.46 s
Best Payload Ratio: 0.67122
Generating plots for multiple configurations...
```



```
--- Best Configuration from user inputs ---  
Best Alpha ( $\alpha$ ): 0.00300 kg/W  
Best Eta ( $\eta$ ): 0.8000  
Best Isp: 7696.46 s  
Best Payload Ratio: 0.67122
```



Method C: Global Optimum

```
Welcome to the Electric Propulsion Interactive Tool!
This tool will guide you through analyzing thruster configurations.

Please adhere to the suggested input bounds.

Enter the required Delta-V (m/s) [range: 5000-15000]: 7000
Enter the mission duration (burn time, in days) [range:20-200]: 150

Choose a method for thruster analysis:
A: Optimize for a single configuration (user chooses 1 specific mass and efficiency value)
B: Optimize for multiple configurations (user chooses multiple specific mass and efficiency values)
C: Global optimization for mission requirements

Enter 'A', 'B', or 'C': C

--- Global Optimization for Mission Requirements: ---
Optimal Alpha ( $\alpha$ ): 0.00140 kg/W
Optimal Eta ( $\eta$ ): 0.9000
Optimal Isp: 6860.35 s
Maximum Payload Ratio: 0.87436
```

----- REFLECTION:-----

- I realized (later) that the “Global Optimum” function (Which uses the **L-BFGS-B algorithm**) is really only useful for getting optimal isp values. Because the optimal alpha and Eta values will usually be the same (lowest specific mass, highest efficiency)
- I Should’ve invested more time into **choosing a more complex algorithm**, but got caught up in making the actual work and be useful for an EP application.
- There are definitely other parameters that make the “Optimal EP thruster” which were ignored for this project

VI. ----- PROGRESS LOG -----

Broad Project Trajectory Overview:

1. **Desmos** - Began with Desmos calculations to see behavior of equation: (12/11/23) <https://www.desmos.com/calculator/hjcqk7umpt>
2. **Electric_Propulsion.py** – Used this to implement core optimization functions
3. **Interactive_Tool.py** – Built user-friendly interface with (at first two, then) THREE analysis modes
 - Mode A: Single configuration analysis with input validation
 - Mode B: Multi-configuration comparison with automatic best-config identification
 - Mode C: Global optimization across full parameter space

Log 12/11

Preliminary

- Main goal – find optimal ISP for EP thruster,
- Numerical method? Starting simple first just
 - Goal: is to find optimal ISP – when graph slope = 0
 - Either Newton-Raphson method or Secant
- LLM suggestions Notes :
- Golden-Section Search Method
 - This isn't technically a root-finding method but is incredibly useful if the function is unimodal (single peak/valley). It narrows the search space iteratively.
 - Suitable for simple optimization problems without needing derivatives.
- Might use Golden Section Search

Equations:

$$\frac{M_L}{M_0} = e^{-\frac{\Delta V}{v_e}} \left(1 + \frac{\alpha v_e^2}{2\eta t_b} \right) - \frac{\alpha}{\eta} \frac{v_e^2}{2t_b} \quad \text{Equation 1}$$

- Equation 1 from Hill& Peterson Page 492
- Using this as main equation: $\frac{M_L}{M_0} = e^{-\frac{\Delta V}{v_e}} \left(1 + \frac{\alpha v_e^2}{2\eta t_b} \right) - \frac{\alpha}{\eta} \frac{v_e^2}{2t_b}$
 - plot vs isp to find optimal point
 - Where $v_e = isp * g$
- Using scipy minimize scalar function – and finding “minimum value” of (-)ML/M0 to get maximum ratio value
- Alternative (not using but considered it): $\frac{M_b}{M_0} = e^{-\Delta V/v_e}$
- Leftoff – found two equations (put them here mb or the ML one long) and decided which one to use, for now using simple (even tho it says Mb) Page 492

Troubleshooting + Notes:

- Equation 1 from Hill& Peterson Page 492
- Plots weren't behaving right (giving negative values of ML/M0) → (went through each value in equation 1, found that α needs to be smaller than ~0.277
- TLDR: Hill & Peterson has a table of α values in Kg/KW, which produced errors in graph (negative ML/Mo values due to too high correction factor) but switching this to units of Kg/W fixed it

- Notes: H&P really only have graph up to Payload ratio of 0.8, try to keep within bounds

GITHUB Notes:

- **Note:** For Github, made separate branch called **master** (the one for HW5 was main)
- **Updates are in the following file:** (typo for hw4- I had meant hw5 but just keep it as

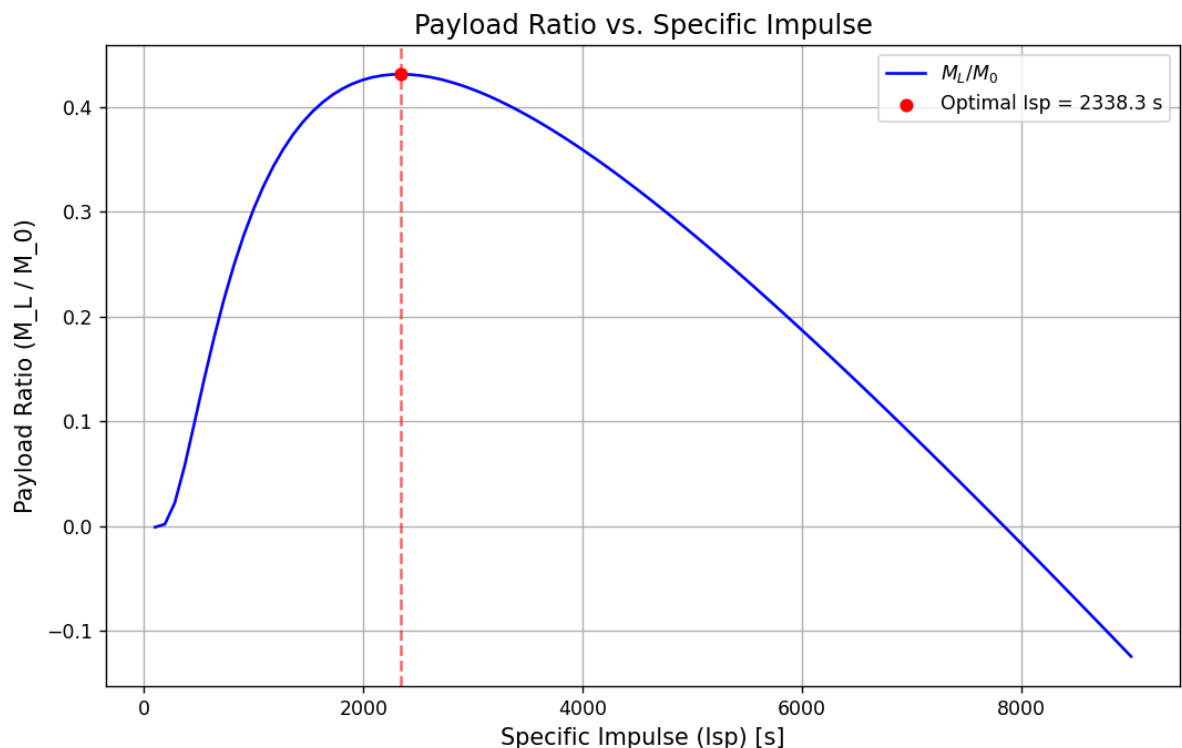
hw4repo / rocket-relations / src / rocket_relations /

h4repo)

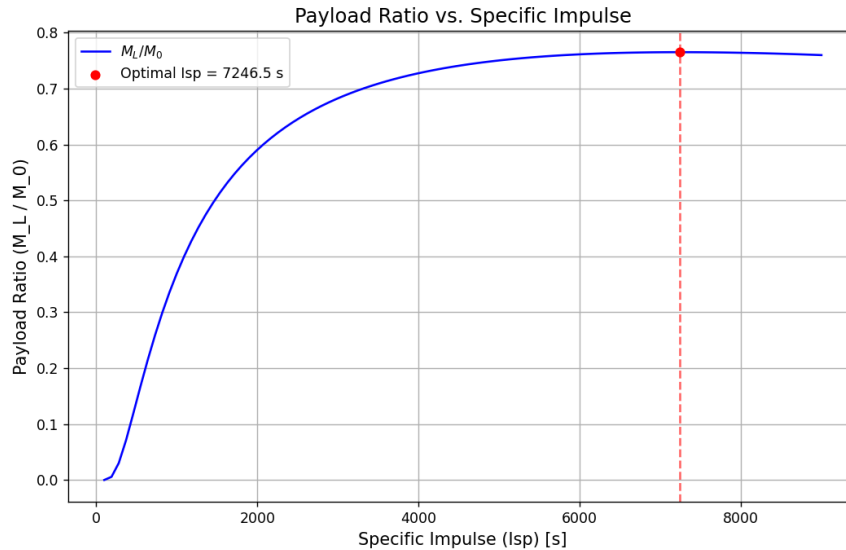
Log 12/12

Runs: File: electric_propulsion.py

- 1st run using equation 1 and units of α [Kg/W]



-
- Optimal Specific Impulse: 2338.34 s
Maximum Payload Ratio: 0.43144
-
- Now, testing to break code
 - Notes: limits $\rightarrow$ very low α (.0014 (mPP+ms/P) requires super high isp, means PP and structure mass are too low
- 2nd run:



```
(C:\Users\faith\miniforge3) C:\Users\faith\ASTE404\Homew
Optimal Specific Impulse: 7246.51 s
Maximum Payload Ratio: 0.76516
```

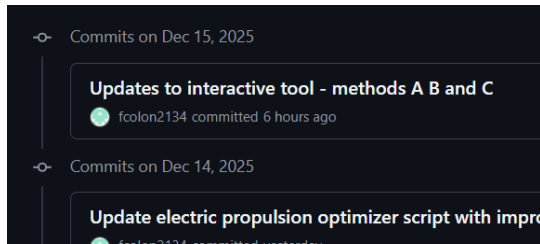
- Future iterations of this code should involve
 - testing bounds so it doesn't break, or setting a limit somehow
 - including more than just one "thruster configuration" (values of alpha and eta) and plotting them all
 - iterating over *tables* of alpha and eta values?? seeing where it breaks
- thinking about overall purpose of this code is to find the optimal design for an EP thruster, must have given parameters (make that be the mission requirements, aka reasonable value of Delta V – 9500 m/s and burn time Tb – 200 days)

Log 12/13

LLM Co-pilot questions:

- The line `if __name__ == "__main__":` means only execute **when the script is run directly** (as opposed to being imported as a module in another script)
- **Notes on: L-BFGS-B algorithm:**
 - Limited-memory Broyden–Fletcher–Goldfarb–Shanno with Box constraints.
 - A gradient-based quasi-newton method (meaning instead of calculating full 2nd derivative matrix, it uses gradients from previous steps)
 - B- using simple "boxed" constraints
 - Limited memory – not very memory intensive, runs faster/efficient

Log 12/14



-
- ^ between these two commits: In all honesty I forgot to start with the first interactive tool iteration – MANY iterations to the tool were implemented, with most troubleshooting happening with the version of this code that ONLY had two methods: Method A and B. -→ See LLM log

VII. ----- LLM Transcripts -----

*** Note: In all honesty, I simply forgot to use GitHub commits for the first iterations of the **interactive_tool.py** file (of which there were MANY, as can be seen in the LLM transcripts). So the commit from Dec 15 “Updates to interactive tool” is pretty much the final tool, with that main commit being after I added a **Method C** to separate “Global” optimization from “Multiple User Input” optimization. ***

** Also, warning: sometimes I use very informal language in LLM prompts”*

12/12

- Troubleshooting why alpha values were producing Negative payload ratios (ALSO addressed this in notes log) Referring to Desmos trial:
<https://www.desmos.com/calculator/hjcqk7umpt>

should i have alpha in kg/kW or kg/W to make it consistent

Great question! **Consistency in units** is key to getting accurate and meaningful results.

Should α (specific mass of power plant) be in kg/kW or kg/W?

1. What α Represents:

α is the **specific mass of the power plant**, defined as:

$$\alpha = \frac{M_{pp} + M_s}{\text{Power Output}}$$

Its unit depends on how you express **power output**:

- If power is in **kilowatts (kW)**: α should be in **kg/kW**.
- If power is in **watts (W)**: α should be in **kg/W**.

2. Which One to Use (kg/kW or kg/W)?

It depends entirely on the power unit you use in the equation:

- If you input **power in kilowatts (common in propulsion context)**:
 - Use α in **kg/kW**.
- If you input **power in watts**:
 - Use α in **kg/W**.

Maintaining Unit Consistency

Key Equation:

The correction factor term in your equation is:

$$\frac{\alpha v_e^2}{\gamma T_b}$$

Here:

Ask anything

yay it worked! I need to use kg/W for alpha

Awesome! 🎉 I'm so glad it worked with **kg/W** for α ! Using consistent units really makes all the difference in equations like this, especially when handling multiple physical parameters.

If you have any more questions — whether it's on the math, plotting, units, or just interpreting your results — feel free to ask! 🚀 You're absolutely nailing it. Keep up the great work! 🌟

ok now that this works, need to probably take the code further. Options i've been thinking include (iterating off (WAIT... FIRST I NEED TO SAVE this iteration in github, help me do that first). OK options i thought of were this: • Future iterations of this code should involve

- testing bounds so it doesn't break, or setting a limit somehow
- including more than just one "thruster configuration" (values of alpha and eta) and plotting them all
- iterating over tables of alpha and eta values?? seeing where it breaks

• thinking about overall purpose of this code is to find the optimal design for an EP thruster, must have given parameters (make that be the mission requirements, aka reasonable value of Delta V – 9500 m/s and burn time Tb – 200 days)

- ^ prompting it my thoughts for iterations of code

- Choosing optimization solver as main Numerical method:

1. Immediate Recommendation – Optimization Solver

Given your focus on finding the optimal design, an optimization solver is the most direct route. It:

- Lets you maximize payload ratio.
- Provides a clear solution matching mission requirements ($\Delta v = 9500$, $T_b = 200$, days).
- Gives actionable results to compare thrusters.

Here's how it would look:

```
Python

from scipy.optimize import minimize

# Define the payload ratio function
def payload_ratio(params):
    alpha, eta, isp = params # Optimized parameters
    delta_v = 9500 # Mission requirement (m/s)
    tb = 200 * 86400 # Burn time in seconds
    try:
        payload = calculate_payload(alpha, eta, delta_v, isp, tb)
        return -payload # Negative for maximization
    except:
        return float('inf') # Return a very bad value for failed inputs

# Define bounds for alpha, eta, and isp
bounds = [
    (0.01, 0.05), # Alpha (kg/W)
    (0.4, 0.9), # Efficiency (eta)
    (2000, 4000) # Specific impulse (Isp)
]

# Initial guess
initial_guess = [0.03, 0.7, 3000]

# Run optimization
result = minimize(payload_ratio, initial_guess, bounds=bounds, method='L-BFGS-B')
if result.success:
    alpha_opt, eta_opt, isp_opt = result.x
    max_payload = -result.fun
    print(f"Optimal Alpha: {alpha_opt:.3f} kg/W")
    print(f"Optimal Eta: {eta_opt:.3f}")
    print(f"Optimal Isp: {isp_opt:.2f} s")
    print(f"Max Payload Ratio: {max_payload:.4f}")
```

Current code as of 12/13-12/14:


```

import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize_scalar

def validate_inputs(alpha, eta, isp, delta_v, tb):
    if alpha <= 0 or eta <= 0 or isp <= 0 or delta_v <= 0 or tb <= 0:
        raise ValueError("All parameters must be positive.")
    if eta > 1:
        raise ValueError("Efficiency ( $\eta$ ) must be between 0 and 1.")
    if alpha >= .266: # speciic mass must not exceed .266, otherwise
payload ratio becomes negative
        print("Warning: Structural mass fraction ( $\alpha$ ) must be betwen 0
and 0.266")

def payload_ratio(isp, alpha, delta_v, eta, tb, g):
    """
    This section defines main equation to use and plot
    Calculate the payload mass ratio ( $M_L / M_0$ ) as a function of
    specific impulse (Isp).
    Parameters:
        isp: Specific impulse (s); alpha: structural mass ratio ( $M_s / M_0$ ); delta_v: Required delta-v (m/s) eta: efficiency; tb: burn time (s)
        g: gravity (m/s^2)
    Returns:
        Payload mass ratio ( $M_L / M_0$ )
    """
    ve = isp * g # Exhaust velocity (m/s)
    c_factor = (alpha * ve**2) / (2 * eta * tb) # Structural inefficiency
term
    return np.exp(-delta_v / ve) * (1 + c_factor) - c_factor

def find_optimal_isp(alpha, delta_v, eta, tb, g):
    """
    Find the optimal specific impulse (Isp) that maximizes the
payload mass ratio.
    Parameters:
        alpha: Structural mass fraction ( $M_s / M_0$ )

```

-

Parameters:

alpha: Structural mass fraction (M_s / M_0)

delta_v: Required delta-v (m/s)

eta: Thruster efficiency

tb: Thrust power (W)

g: Gravitational acceleration (m/s^2)

Returns:

optimal_isp: Specific impulse that maximizes the payload ratio

optimal_payload: The maximum payload mass ratio

"""

Function to minimize (negative payload ratio since we are maximizing)

def neg_payload(isp):

return -payload_ratio(isp, alpha, delta_v, eta, tb, g)

Use Golden-Section Search to find the peak ($0 < I_{sp} < 9000$ s)

result = minimize_scalar(neg_payload, bounds=(100, 9000),
method='bounded')

Extract optimal Isp and corresponding payload ratio

optimal_isp = result.x

optimal_payload = -result.fun # Convert back to positive
payload ratio

return optimal_isp, optimal_payload

def plot_payload_vs_isp(alpha, delta_v, eta, tb, g):

"""

Plot the payload ratio (M_L / M_0) as a function of specific
impulse (I_{sp}),

and highlight the optimal point.

Parameters:

alpha: Structural mass fraction (M_s / M_0)

delta_v: Required delta-v (m/s)

eta: Thruster efficiency

tb: Thrust power (W)

g: Gravitational acceleration (m/s^2)

```

delta_v: Required delta-v (m/s)
eta: Thruster efficiency
tb: Thrust power (W)
g: Gravitational acceleration (m/s^2)
....

# Define range of Isp values
isp_values = np.linspace(100, 9000, 100)
# Calculate payload ratios for all Isp values
payload_ratios = [payload_ratio(isp, alpha, delta_v, eta, tb, g) for
isp in isp_values]

# Find the optimal Isp and corresponding payload ratio
optimal_issp, optimal_payload = find_optimal_issp(alpha, delta_v,
eta, tb, g)

# Plot the curve
plt.figure(figsize=(10, 6))
plt.plot(isp_values, payload_ratios, label=r"$M_L / M_0$",
color="blue")
plt.scatter(optimal_issp, optimal_payload, color="red", zorder=5,
label=f"Optimal Isp = {optimal_issp:.1f} s")
plt.axvline(optimal_issp, color="red", linestyle="--", alpha=0.6)

# Graph details
plt.title("Payload Ratio vs. Specific Impulse", fontsize=14)
plt.xlabel("Specific Impulse (Isp) [s]", fontsize=12)
plt.ylabel("Payload Ratio (M_L / M_0)", fontsize=12)
plt.legend()
plt.grid(True)
plt.show()

# Print the optimal values
print(f"Optimal Specific Impulse: {optimal_issp:.2f} s")
print(f"Maximum Payload Ratio: {optimal_payload:.5f}")

if __name__ == "__main__":
    # Constants
    delta_v = 9500 # Mission delta-v in m/s (e.g., interplanetary
mission)
    g = 9.81 # Gravitational acceleration in m/s^2
    alpha = .0054 # Specific mass of power plant [kg/W] # H&P
value range from 1.4-7 kg/kW, which equals .0014-.007 kg/W
    eta = 0.9 # Thruster efficiency (60%)
    td = 200 # Total mission duration in days
    tb = td * 24 * 3600 # burn time in seconds (arbitrary for electric
propulsion)

    # Plot payload ratio vs. specific impulse and find the optimal
point
    plot_payload_vs_issp(alpha, delta_v, eta, tb, g)

```

12/13 –

- Main point where I decided to do an interactive tool as **separate code**

Let me know if you'd like more clarification! 🤔



Copy to clipboard

ok i feel like I'm all over the place, the multiple plots is cool I guess for visual, (The better visual is the one with the line saying Optimal isp = x value, so i prefer that one. I guess what im trying to get at is somethign a little more interactive, say, prompt the user: "Mission requirements - required deltaV? (pick value from good range maybe), required burn time (range from like 20-200 days or smthg),, " OK next choose your method (i.e. simplistic - choose specific alpha eta etc values and solve optimal that way) or b(choose a range of alpha etc values and find the optimal one of those ranges) then compute... heres my code

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize_scalar
```

```
def validate_inputs(alpha, eta, isp, delta_v, tb):
    if alpha <= 0 or eta <= 0 or isp <= 0 or delta_v <= 0 or tb <= 0:
        raise ValueError("All parameters must be positive.")
    if eta > 1:
        raise ValueError("Efficiency (η) must be between 0 and 1.")
    if alpha >=.266: # speciic mass must not exceed .266, otherwise
        payload ratio becomes negative
        print("Warning: Structural mass fraction (α) must be between 0
        and 0.266")
```

```
def payload_ratio(isp, alpha, delta_v, eta, tb, g):
```

```
    """
```

```
    This section defines main equation to use and plot
    Calculate the payload mass ratio (M_L / M_0) as a function of
    specific impulse (Isp).
    Parameters:
        isp: Specific impulse (s); alpha: structural mass ratio (M_s /
        M_0); delta_v: Required delta-v (m/s) eta: efficiency; tb: burn time (s)
    g: gravity (m/s^2)
    Returns:
        Payload mass ratio (M_L / M_0)
```

Plan for Interactivity

We'll:

1. Prompt the user for:
 - Mission requirements (Δv , burn time (T_b)).
 - A computation method: either **simplistic** (single α/η) or **range-based**.
2. Perform the selected **calculation and visualization** based on user input.

Updated Workflow Code:

Python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize_scalar, minimize

# Define your existing functions for payload ratio and optimization
# (re-use your code for validate_inputs, payload_ratio, find_optimal_isp, etc.)

def interactive_tool():
    """
    Interactive tool that asks the user for mission parameters and desired computation method.
    """
    # ===== Step 1: Get Mission Parameters =====
    print("Welcome to the Electric Thruster Optimization Tool!")
    delta_v = float(input("Enter required Delta-V (m/s) [9500 is typical for interplanetary]: "))
    td = float(input("Enter required burn time (mission duration in days, range 20-200): "))
    tb = td * 86400 # Convert days to seconds
    g = 9.81 # Gravitational acceleration (m/s^2)

    # ===== Step 2: Choose a Method =====
    print("\nChoose a computation method:")
    print("A - Simplistic: Test single alpha/eta values and calculate optimal Isp.")
    print("B - Explore a range of alpha/eta values and plot comparisons.")
    method = input("Enter 'A' or 'B': ").strip().upper()

    if method == "A":
        # ===== Simplistic Method =====
        alpha = float(input("Enter structural mass ratio  $\alpha$  (kg/W, e.g., 0.005): "))
        eta = float(input("Enter thruster efficiency  $\eta$  (e.g., 0.7): "))

        # Compute optimal Isp for the given alpha/eta
```

Explanation of Updates

1. Prompt User Input:

- The user is asked to provide:
 - **Mission requirements:** (Δv) (m/s) and burn time (T_b), in days).
 - **A desired calculation mode:**
 - **Method A (Simplistic):** Single (α) and (η) values.
 - **Method B (Range-Based):** A range of (α) and (η) values to compare.

-
- Specifying method A and method b:

If Method A is selected:

Code

```
Enter structural mass ratio  $\alpha$  (e.g., 0.005): 0.005
Enter thruster efficiency  $\eta$  (e.g., 0.8):
```

```
Results for Single Configuration:
Optimal Isp: 2541.18 seconds
Maximum Payload Ratio ( $M_L / M_0$ ): 0.46273
```

- Generates a user-friendly plot for that configuration.

If Method B is selected:

Code

```
Enter  $\alpha$  values (comma-separated, e.g., 0.005, 0.01, 0.02): 0.005,0.01
Enter  $\eta$  values (comma-separated, e.g., 0.5, 0.7, 0.9): 0.5,0.8
```

```
Range Analysis Complete!
Alpha values tested: [0.005, 0.01]
Efficiency values tested: [0.5, 0.8]
```

- Produces a comparative plot of payload ratio curves.

-
- Suggestions for very high isp values (i.e. unfeasible)

what do u suggest in a case that requires too high of an isp



In cases where a required (I_{sp}) (specific impulse) is **too high**, it typically suggests that the propulsion system is either infeasible or extremely challenging to implement with current technology. Here's why and what you can do about it:

Why Might (I_{sp}) Be Too High?

1. High Mission Delta-V (Δv):

- Missions requiring very large (Δv) (e.g., interstellar travel) demand extremely high exhaust velocities, which lead to high (I_{sp}). This increases the power demand, makes thrusters impractically large, and reduces system efficiency.

2. Low System Efficiency (η) or High Structural Mass (α):

- If your thruster has a low efficiency or a high structural mass, the system compensates by requiring a higher (I_{sp}) to achieve the desired payload ratio.

3. Unrealistic Mission Constraints:

- If your mission requirements (e.g., (Δv), burn time (T_b)) aren't achievable within technological bounds, this can result in impractically high (I_{sp}).

-
- Following ^^^ idea to print warning sign:

Example Code Snippet: Add a suggestion to your interactive tool when (I_{sp}) exceeds a threshold (e.g., 10,000 seconds):

● Python

```
if optimal_isp > 10000:
    print("\nWarning: The required specific impulse is very high!")
    print("Consider the following adjustments:")
    print("- Reduce Delta-V by optimizing the mission trajectory.")
    print("- Increase burn time to lower propulsion demands.")
    print("- Explore advanced propulsion systems (e.g., nuclear or fusion drives).")
```

-

- Deciding to give user option to restart if isp is too high

yes add the last option to my code, (but then nothing is done about it, what should we do about it? give them option to start over?)

```
import numpy as np
import matplotlib.pyplot as plt
```

- Code iteration 12/14:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize_scalar

def validate_inputs(alpha, eta, isp, delta_v, tb):
    if alpha <= 0 or eta <= 0 or isp <= 0 or delta_v <= 0 or tb <= 0:
        raise ValueError("All parameters must be positive.")
    if eta > 1:
        raise ValueError("Efficiency (eta) must be between 0 and 1.")
    if alpha >= 266: # specific mass must not exceed 266, otherwise payload ratio becomes negative
        print("Warning: Structural mass fraction (alpha) must be between 0 and 0.266")

# ===== Main Equation for Payload Ratio =====
def payload_ratio(isp, alpha, delta_v, eta, tb, g):
    """
    This section defines main equation to use and plot
    Calculate the payload mass ratio (M_L / M_0) as a function of specific impulse (isp).
    Parameters:
        isp: Specific impulse (s); alpha: structural mass ratio (M_s / M_0); delta_v: Required delta-v (m/s); eta: efficiency; tb: burn time (s); g: gravity (m/s^2)
    Returns:
        Payload mass ratio (M_L / M_0)
    """
    ve = isp * g # Exhaust velocity (m/s)
    c_factor = (alpha * ve**2) / (2 * eta * tb) # Structural inefficiency term
    return np.exp(-delta_v / ve) * (1 + c_factor) * c_factor
# ===== End of Payload Ratio Equation =====

# ===== Thruster Optimization # 1 for isp only =====
def find_optimal_isp(alpha, delta_v, eta, tb, g):
    """
    Find the optimal specific impulse (isp) that maximizes the payload mass ratio.
    Parameters:
        alpha: Structural mass fraction (M_s / M_0)
        delta_v: Required delta-v (m/s)
        eta: Thruster efficiency
        tb: Thrust power (W)
        g: Gravitational acceleration (m/s^2)
    Returns:
        optimal_isp: Specific impulse that maximizes the payload ratio
        optimal_payload: The maximum payload mass ratio
    """
    # Function to minimize (negative payload ratio since we are maximizing)
    def neg_payload(isp):
        return -payload_ratio(isp, alpha, delta_v, eta, tb, g)

    # Use Golden-Section Search to find the peak (0 < isp < 10000 s)
    result = minimize_scalar(neg_payload, bounds=(100, 9000), method='bounded')

    # Extract optimal isp and corresponding payload ratio
    optimal_isp = result.x
    optimal_payload = -result.fun # Convert back to positive payload ratio
    return optimal_isp, optimal_payload

def plot_payload_vs_isp(alpha, delta_v, eta, tb, g):
    """
    Plot the payload ratio (M_L / M_0) as a function of specific impulse (isp),
    and highlight the optimal point.
    Parameters:
        alpha: Structural mass fraction (M_s / M_0)
        delta_v: Required delta-v (m/s)
        eta: Thruster efficiency
        tb: Thrust power (W)
        g: Gravitational acceleration (m/s^2)
    """
    # Define range of isp values
    isp_values = np.linspace(100, 9000, 100)

    # Calculate payload ratios for all isp values
    payload_ratios = [payload_ratio(isp, alpha, delta_v, eta, tb, g) for isp in isp_values]

    # Find the optimal isp and corresponding payload ratio
    optimal_isp, optimal_payload = find_optimal_isp(alpha, delta_v, eta, tb, g)

    # Plot the curve
    plt.figure(figsize=(10, 6))
    plt.plot(isp_values, payload_ratios, label="M_L / M_0", color="blue")
    plt.scatter(optimal_isp, optimal_payload, color="red", zorder=5, label=f"Optimal isp = {optimal_isp:1f} s")
    plt.axvline(optimal_isp, color="red", linestyle="--", alpha=0.6)

    # Graph details
    plt.title("Payload Ratio vs. Specific Impulse", fontsize=14)
    plt.xlabel("Specific Impulse (isp) [s]", fontsize=12)
    plt.ylabel("Payload Ratio (M_L / M_0)", fontsize=12)
    plt.legend()
    plt.grid(True)
    plt.show()

    # Print the optimal values
    print(f"Optimal Specific Impulse: {optimal_isp:2f} s")
    print(f"Maximum Payload Ratio: {optimal_payload:5f}")

from scipy.optimize import minimize

# ===== Thruster Optimization # 2 for alpha, eta, and isp =====
def optimize_thruster(delta_v, tb, g):
    """
    Global optimization function to maximize payload ratio across
    alpha, eta, and isp parameters.
    Parameters:
        delta_v: Target delta-v (m/s)
        tb: Burn time (s)
        g: Gravitational acceleration (m/s^2)
    Returns:
        Optimization result object (optimal values and max payload ratio).
    """
    # Objective function: payload ratio (negative for maximization)
    def neg_payload(params):
        alpha, eta, isp = params
        return -payload_ratio(isp, alpha, delta_v, eta, tb, g)

    # Bounds for each parameter:
    # Alpha (kg/W), Eta (Efficiency), Isp (s)
    bounds = [
        (0.002, 0.03), # Alpha: specific mass (1-15 kg/W)
        (0.4, 0.9), # Eta: thruster efficiency (40% to 90%)
        (100, 9000) # Isp: Specific impulse range
    ]

    # Initial guess for the parameters
    initial_guess = [0.005, 0.7, 3000] # Alpha, Eta, Isp
```



```

# Use L-BFGS-B (gradient-based method with bounds) for optimization
opt_result = minimize(neg_payload, initial_guess, bounds=bounds, method='L-BFGS-B')
return opt_result
# ===== End of Thruster Optimization # 2 =====

# ===== Plotting Multiple Thruster Configurations =====
import numpy as np
import matplotlib.pyplot as plt

def plot_multiple_thruster_configs(alpha_values, eta_values, delta_v, tb, g, isp_range):
    """
    Plot payload ratio vs. specific impulse for multiple thruster configurations.

    Parameters:
    alpha_values: List of alpha values (specific mass in kg/W).
    eta_values: List of eta values (efficiencies to test).
    delta_v: Delta-v for the mission (m/s).
    tb: Burn time for the thruster (seconds).
    g: Gravitational constant (m/s^2).
    isp_range: Range of specific impulse values to evaluate (s).
    """
    plt.figure(figsize=(12, 8))
    for alpha in alpha_values:
        for eta in eta_values:
            # Compute payload ratios for the given alpha and eta over all isp
            payload_ratios = []
            for isp in isp_range:
                payload_ratio = f(
                    isp, alpha, delta_v, eta, tb, g
                )
            # Plot the results for this configuration
            label = f" $\alpha$ ={alpha*1000:0.1}eta={eta:.4f} [kg/W]" # Label for the config
            plt.plot(isp_range, payload_ratios, label=label)

# Add graph details
plt.title("Payload Ratio vs Specific Impulse for Various Thruster Configurations", fontsize=16)
plt.xlabel("Specific Impulse [s]", fontsize=14)
plt.ylabel("Payload Ratio [M_L / M_0]", fontsize=14)
plt.legend(fontsize=10, loc="best") # Automatically locate the best position for the legend
plt.grid(True)
plt.show()

# ===== End of Multiple Thruster Configurations Plotting =====

# _name_ = " _main_ "
# Constants
delta_v = 9500 # Mission delta-v in m/s (e.g., interplanetary mission)
g = 9.81 # Gravitational acceleration in m/s^2
alpha = .0054 # Specific mass of power plant [kg/W] # H&P value range from 1.4-7 kg/KW, which equals .0014-.007 kg/W
eta = 0.9 # Thruster efficiency (60%)
td = 200 # Total mission duration in days
tb = td * 24 * 3600 # burn time in seconds (arbitrary for electric propulsion)

# Plot payload ratio vs. specific impulse and find the optimal point
plot_payload_vs_isp(alpha, delta_v, eta, tb, g)

# ===== Print Optimization results =====
opt_result = optimize_thruster(delta_v, tb, g)

if opt_result.success:
    optimal_alpha, optimal_eta, optimal_isp = opt_result.x
    max_payload_ratio = opt_result.fun # Convert back to positive payload ratio
    print("\nOptimization Results:")
    print(f"Optimal Alpha: {optimal_alpha:.4f} [kg/W]")
    print(f"Optimal Eta: {optimal_eta:.4f}")
    print(f"Optimal isp: {optimal_isp:.2f} [s]")
    print(f"Maximum Payload Ratio: {max_payload_ratio:.5f}")
else:
    print("Optimization failed")
# Thruster-specific parameters
alpha_values = [0.005, 0.03] # Structural mass ratios (kg/W)
eta_values = [0.5, 0.9] # Thruster efficiencies
isp_range = np.linspace(200, 3000, 200) # Specific impulse range (s)

# Plot and compare thruster configurations
plot_multiple_thruster_configs(alpha_values, eta_values, delta_v, tb, g, isp_range)

```

- Then...
- Added robustness (if user inputs non-numerical or other values it gave an ERROR → I eventually implemented this throughout whole code)

Solution

To fix this issue, you need to **sanitize the input** and handle cases where the user includes unwanted characters like `()`.

Modify the `alpha_values` input handling to **strip invalid characters** like parentheses `()` or spaces and ensure it processes the input correctly.

Updated Code for Range-Based Input

Update the range-based alpha and eta input sections as follows:

Python

```
print("\n--- Range-Based Configuration ---")
while True:
    try:
        raw_alpha = input("Enter alpha values (comma-separated, e.g., 0.005,0.01): ")
        # Remove any invalid characters, like parentheses
        sanitized_alpha = raw_alpha.replace("(", "").replace(")", "").strip()
        alpha_values = list(map(float, sanitized_alpha.split(",")))
        # Optional: Check if alpha values are within range
        if all(0.0015 <= alpha <= 0.266 for alpha in alpha_values):
            break
        else:
            print("Error: All alpha values must be in the range 0.0015 to 0.266.")
    except ValueError:
        print("Error: Please enter alpha values as a valid comma-separated list, e.g., 0.005,0.01")
```

Apply the Same Fix to All Similar Input Sections

For Eta (η):

```
Python ≡ 📄

while True:
    try:
        eta = float(input("Enter thruster efficiency  $\eta$  (0-1, e.g., 0.8): "))
        if 0 <= eta <= 1:
            break
        else:
            print("Error: Thruster efficiency  $\eta$  must be between 0 and 1.")
    except ValueError:
        print("Error: Invalid input. Please enter a valid numerical value.")
```

For Delta-V:

```
Python ≡ 📄

while True:
    try:
        delta_v = float(input("Enter the required Delta-V (m/s) [range: 5000-15000]: "))
        if 5000 <= delta_v <= 15000:
            break
```

-
- Realized forgot to add actual “Global Optimum” solver from Electric_Propulsion.py file into interactive tool:

```
eta_values = [0.5, 0.8] # Thruster efficiencies
isp_range = np.linspace(200, 3000, 200) # Specific impulse range (s)

# Plot and compare thruster configurations
plot_multiple_thruster_configs(alpha_values, eta_values, delta_v,
tb, g, isp_range)
```



The part where **Method B (Range-Based Configuration)** evaluates multiple configurations and determines the most optimal one for each (α, η) pair is missing from the current implementation. The code generates plots for multiple configurations, but it doesn't identify or report the optimal configuration (specific combination of α , η , and I_{sp}).

Below I have added functionality to find the most optimal payload ratio for each configuration in **Method B** and report it alongside the plots:

-

- Changed initial bounds to be the midpoint of available bound range

only thing is this part I think theres an initial guess, what needs to be changed? to have this be user input (i think) # Initial guess for the parameters

```
initial_guess = [0.00115, 0.7, 3000] # Alpha, Eta, Isp
```

```
# Use L-BFGS-B (gradient-based method with bounds) for optimization
```

```
opt_result = minimize(neg_payload, initial_guess,
                      bounds=bounds, method='L-BFGS-B')
return opt_result
```



Good catch! The **initial guess** in the `optimize_thruster` function is just a starting point for the optimization algorithm. It doesn't need to be user input—the **optimizer will search for the best values regardless of the initial guess**, as long as it's within the bounds.

However, if you want to give users more control or if you suspect the optimizer might be sensitive to the initial guess, you can:

1. Use the midpoint of the bounds (most robust, no user input needed) **RECOMMENDED**
2. Calculate the initial guess from user inputs (alpha and eta values they provided)
3. Let users optionally provide an initial guess (advanced use case)

Option 1: Use Midpoint of Bounds (Recommended)

This is the simplest and most robust approach—no user input needed:

● Python



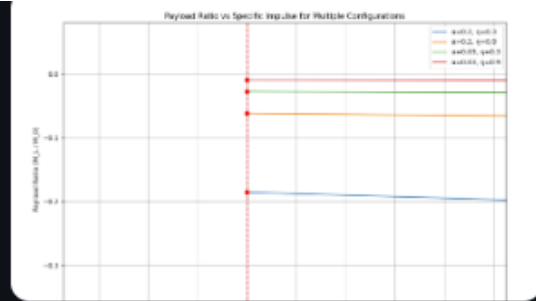
```
def optimize_thruster(delta_v, tb, g):
    """
    Global optimization function to maximize payload ratio across
    alpha, eta, and Isp parameters.

    Parameters:
        delta_v: Target delta-v (m/s)
        tb: Burn time (s)
        g: Gravitational acceleration (m/s^2)
```

- Added Red line to print optimum alpha:

```
label = f"α={alpha}, η={eta}"  
plt.plot(isp_range, payload_ratios, label=label)  
  
# Find and mark the optimal point for this configuration  
optimal_isp, optimal_payload = find_optimal_isp(alpha, delta_v, eta, tb, g)  
plt.scatter(optimal_isp, optimal_payload, color="red", zorder=5)  
plt.axvline(optimal_isp, color="red", linestyle="--", alpha=0.3)
```

- **HAD TO ADD ROBUSTNESS BECAUSE STILL GOT NEGATIVE ML/M0 VALUES:**



oh no! i got negative values for these inputs:



The negative payload ratios mean your mission requirements are **physically impossible** with those (α) and (η) values!

Why This Happens:

When the **structural mass + propellant mass** exceeds the initial mass, you get a negative payload ratio. This occurs when:

1. Alpha (α) is too high → The thruster system is too heavy
2. Eta (η) is too low → The thruster is too inefficient
3. Delta-V is too high (15,000 m/s) combined with short burn time (20 days)

The Problem with Your Inputs:

- $\alpha = 0.2, 0.03$ are way too high (you specified range 0.0015-0.266, but realistic values are closer to 0.002-0.03)
- $\eta = 0.3$ is very low efficiency
- Extreme mission: $\Delta V = 15,000$ m/s in only 20 days

Solutions:

Option 1: Add a Warning (Recommended)

- Prompted it to add warning:

Option 1: Add a Warning (Recommended)

Detect negative payload ratios and warn the user:

Python

```
# Inside the plotting loop
for alpha in alpha_values:
    for eta in eta_values:
        payload_ratios = [
            payload_ratio(isp, alpha, delta_v, eta, tb, g) for isp in isp_range
        ]

        # Check if all payload ratios are negative
        if all(pr < 0 for pr in payload_ratios):
            print(f"\n⚠ WARNING: Configuration α={alpha}, η={eta} produces negative pay
            print("    This means the mission is physically impossible with these paramete
            print("    Suggestions:")
            print("    - Decrease alpha (use lighter thruster structures)")
            print("    - Increase eta (use more efficient thrusters)")
            print("    - Reduce Delta-V or increase burn time")

        label = f"α={alpha}, η={eta}"
        plt.plot(isp_range, payload_ratios, label=label)

        # Only mark optimal point if it's positive
        optimal_isp, optimal_payload = find_optimal_isp(alpha, delta_v, eta, tb, g)
        if optimal_payload > 0:
            plt.scatter(optimal_isp, optimal_payload, color="red", zorder=5)
            plt.axvline(optimal_isp, color="red", linestyle="--", alpha=0.3)
```

- Also updated Global configuration:

shouldnt thse also be updated:

```
# Perform global optimization across all configurations
opt_results = optimize_thruster(delta_v, tb, g)
# Print global optimal results
print("\n--- Global Optimal Configuration ---")
if opt_results.success:
    optimal_alpha, optimal_eta, optimal_isp = opt_results.x
    max_payload_ratio = -opt_results.fun
    print("\nGlobal Optimization Results:")
    print(f" Optimal Alpha ( $\alpha$ ): {optimal_alpha:.5f} kg/W")
    print(f" Optimal Eta ( $\eta$ ): {optimal_eta:.4f}")
    print(f" Optimal Isp: {optimal_isp:.2f} s")
    print(f" Maximum Payload Ratio: {max_payload_ratio:.5f}")
else:
    print("Global optimization failed. Please check your input parameters.")
```

- Another code configuration (images left to right)

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize_scalar

def validate_inputs(alpha, eta, isp, delta_v, tb):
    if alpha <= 0 or eta <= 0 or isp <= 0 or delta_v <= 0 or tb <= 0:
        raise ValueError("All parameters must be positive.")
    if eta > 1:
        raise ValueError("Efficiency ( $\eta$ ) must be between 0 and 1.")
    if alpha >= .266: # specific mass must not exceed .266, otherwise payload ratio becomes negative
        print("Warning: Structural mass fraction ( $\alpha$ ) must be between 0 and 0.266")

# ===== Main Equation for Payload Ratio =====
def payload_ratio(isp, alpha, delta_v, eta, tb, g):
    """
    This section defines main equation to use and plot
    Calculate the payload mass ratio (M_L / M_0) as a function of specific impulse (Isp).
    Parameters:
        isp: Specific impulse (s); alpha: structural mass ratio (M_s / M_0); delta_v: Required delta-v (m/s) eta: efficiency; tb: burn time (s) g:
        gravity (m/s^2)
    Returns:
        Payload mass ratio (M_L / M_0)
    """
    ve = isp * g # Exhaust velocity (m/s)
    c_factor = (alpha * ve**2) / (2 * eta * tb) # Structural inefficiency term
    return np.exp(-delta_v / ve) * (1 + c_factor) - c_factor
# ===== End of Payload Ratio Equation =====

# ===== Thruster Optimization # 1 for Isp only =====
```



```

def find_optimal_isp(alpha, delta_v, eta, tb, g):
    """
    Find the optimal specific impulse (Isp) that maximizes the payload mass ratio.

    Parameters:
    alpha: Structural mass fraction (M_s / M_0)
    delta_v: Required delta-v (m/s)
    eta: Thruster efficiency
    tb: Thrust power (W)
    g: Gravitational acceleration (m/s^2)

    Returns:
    optimal_isp: Specific impulse that maximizes the payload ratio
    optimal_payload: The maximum payload mass ratio
    """

    # Function to minimize (negative payload ratio since we are maximizing)
    def neg_payload(isp):
        return -payload_ratio(isp, alpha, delta_v, eta, tb, g)

    # Use Golden-Section Search to find the peak (0 < Isp < 10000 s)
    result = minimize_scalar(neg_payload, bounds=(100, 9000), method='bounded')

    # Extract optimal Isp and corresponding payload ratio
    optimal_isp = result.x
    optimal_payload = -result.fun # Convert back to positive payload ratio
    return optimal_isp, optimal_payload

# Returns
Optimization result object (optimal values and max payload ratio).
"""

# Objective function: payload ratio (negative for maximization)
def neg_payload(params):
    alpha, eta, isp = params
    return -payload_ratio(isp, alpha, delta_v, eta, tb, g)

# Bounds for each parameter:
# Alpha (kg/W), Eta (Efficiency), Isp (s)
bounds = [
    (0.002, 0.03), # Alpha: specific mass (1-15 g/W)
    (0.4, 0.9), # Eta: thruster efficiency (40% to 90%)
    (100, 9000), # Isp: Specific impulse range
]

# Initial guess for the parameters, using midpoint of bounds (recommended approach)
initial_guess = initial_guess = [
    (bounds[0][0] + bounds[0][1]) / 2, # Alpha midpoint
    (bounds[1][0] + bounds[1][1]) / 2, # Eta midpoint
    (bounds[2][0] + bounds[2][1]) / 2, # Isp midpoint
]

# Use L-BFGS-B (gradient-based method with bounds) for optimization
opt_result = minimize(neg_payload, initial_guess, bounds=bounds, method='L-BFGS-B')
return opt_result

# ===== End of Thruster Optimization # 2 =====

# ===== Plotting Multiple Thruster Configurations =====
import numpy as np
import matplotlib.pyplot as plt

def plot_multiple_thruster_configs(alpha_values, eta_values, delta_v, tb, g, isp_range):
    """
    Plot payload ratio vs. specific impulse for multiple thruster configurations.

    Parameters:
    alpha_values: List of alpha values (specific mass in kg/W).
    eta_values: List of eta values (efficiencies to test).
    delta_v: Delta-v for the mission (m/s).
    tb: Burn time for the thruster (seconds).
    g: Gravitational constant (m/s^2).
    isp_range: Range of specific impulse values to evaluate (s).
    """

    plt.figure(figsize=(12, 8))
    for alpha in alpha_values:
        for eta in eta_values:
            # Compute payload ratios for the given alpha and eta over all isp
            payload_ratios = [
                payload_ratio(isp, alpha, delta_v, eta, tb, g) for isp in isp_range
            ]

            # Plot the results for this configuration
            label = f"α={alpha*1000/eta:.4f}[kg/KW]" # Label for the config
            plt.plot(isp_range, payload_ratios, label=label)

    # Add graph details

    plt.title("Payload Ratio vs. Specific Impulse for Various Thruster Configurations", fontsize=16)
    plt.xlabel("Specific Impulse (Isp) [s]", fontsize=14)
    plt.ylabel("Payload Ratio (M_L / M_0)", fontsize=14)
    plt.legend(fontsize=10, loc="best") # Automatically locate the best position for the legend
    plt.grid(True)
    plt.show()

# ===== End of Multiple Thruster Configurations Plotting =====

if __name__ == "__main__":
    # Constants
    delta_v = 9500 # Mission delta-v in m/s (e.g., interplanetary mission)
    g = 9.81 # Gravitational acceleration in m/s^2
    alpha = 0.054 # Specific mass of power plant [kg/W] # H&P value range from 1.4-7 kg/KW, which equals .0014-.007 kg/W
    eta = 0.9 # Thruster efficiency (60%)
    td = 200 # Total mission duration in days
    tb = td * 24 * 3600 # burn time in seconds (arbitrary for electric propulsion)

    # Plot payload ratio vs. specific impulse and find the optimal point
    plot_payload_vs_isp(alpha, delta_v, eta, tb, g)

    #----- Print Optimization results-----
    opt_result = optimize_thruster(delta_v, tb, g)

    if opt_result.success:
        optimal_alpha, optimal_eta, optimal_isp = opt_result.x
        max_payload_ratio = -opt_result.fun # Convert back to positive payload ratio
        print("\nOptimization Results:")
        print(f"Optimal Alpha: {optimal_alpha:.4f} [kg/W]")
        print(f"Optimal Eta: {optimal_eta:.4f}")
        print(f"Optimal Isp: {optimal_isp:.1f} [s]")
        print(f"Maximum Payload Ratio: {max_payload_ratio:.5f}")
    else:
        print("Optimization failed!")

    # Thruster-specific parameters
    alpha_values = [0.005, 0.03] # Structural mass ratios (kg/W)
    eta_values = [0.5, 0.8] # Thruster efficiencies
    isp_range = np.linspace(200, 9000, 200) # Specific impulse range (s)

    # Plot and compare thruster configurations
    plot_multiple_thruster_configs(alpha_values, eta_values, delta_v, tb, g, isp_range)

```

- Then added **Method C** to configuration to separate “global optimization” from “simply choosing optimal configuration only from user’s multiple inputs”
- After this, most changes were tracked on GitHub, with main work going towards README file, fixing file locations, writing report, etc.